Modeling Kubernetes invariants in Lean

Sachin Singh

https://github.com/sachinkumarsingh092/lean-rl/tree/master/lean_rl/lean

A routine scale-down

```
$ kubectl scale deploy/web --replicas=1
```

- The webhook admits it and the deployment scales down.
- `web` had a PDB with `minAvailable: 2`.
- The PDB controller logs the violation, but only after the cluster has already lost the guarantee.
- Customers see the blip and the pager fires.

The thing that's missing is a model of what the cluster will look like next.

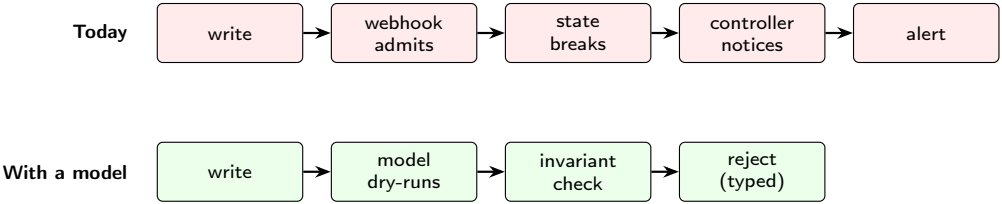
Why this keeps happening

- Validating webhooks evaluate the **incoming object**, not the **post-state of the cluster**.
- Cluster-wide invariants are enforced by **controllers, after the write lands**:
  - PDB `minAvailable`
  - node CPU and memory capacity
  - anti-affinity and topology spread
  - ResourceQuota
- That is reconciliation, not prevention. It works eventually, but your SLO does not get to wait that long.

What is Lean

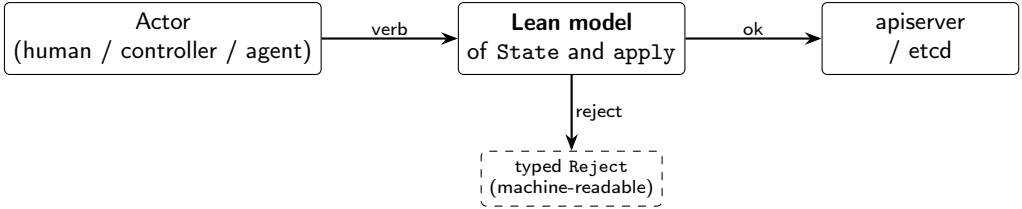
- Lean is a functional programming language whose compiler is also a proof checker.
- You write ordinary definitions: data structures, functions, predicates.
- You can also state theorems about those definitions and have the compiler check the proof.
- Same language, same file. The thing you prove about is the thing that runs.
- No model checker, no separate spec language. Just code, plus a proof obligation.

Today vs. what we want



The bad write never lands. The operator sees a structured reason instead of a postmortem.

The model



`apply : State → ApiVerb → State` is a function in Lean.
Once it is defined, post-state checks and proofs are just questions you can ask about it.

“Isn’t this Gatekeeper / Kyverno / CEL?”

	Gatekeeper / Kyverno / CEL	**Lean model**
Evaluates	Incoming object + cached refs	Post-state $apply(s, v)$
Coverage	What the policy happens to check	All states, all verbs
Backed by	Reviewed policy code	Kernel-checked theorem

This is a different category. Gatekeeper and Kyverno are policy engines. What we have is a formal model of cluster transitions, which you can interrogate with a policy engine, an admission webhook, an RL judge, or a TLA+-style review.

The model (1/3): state and verbs

```
structure State where
  pods        : List Pod
  nodes       : List Node
  deployments : List Deployment

inductive ApiVerb
  | createPod | createDeployment
  | updateDeployment | scaleDeployment | deletePodByKey

def apply : State → ApiVerb → State    -- pure, deterministic
```

This is a small slice of the API, expressed as plain data plus a single function.

The model (2/3): invariants are a registry

```
def invariants : List (InvariantKind × (State → Bool)) :=
  [ (.pdb,          pdbRespected),
    (.capacity,     capacityRespected),
    (.antiAffinity, antiAffinityRespected) ]

def safe (s : State) : Bool := invariants.all ·(.2 s)
```

- Each invariant is just one decidable predicate on `State`.
- **Adding your own is one line in this list.**
- Topology spread, quota, and custom org-specific rules all have the same shape.

The model (3/3): what it lets us prove

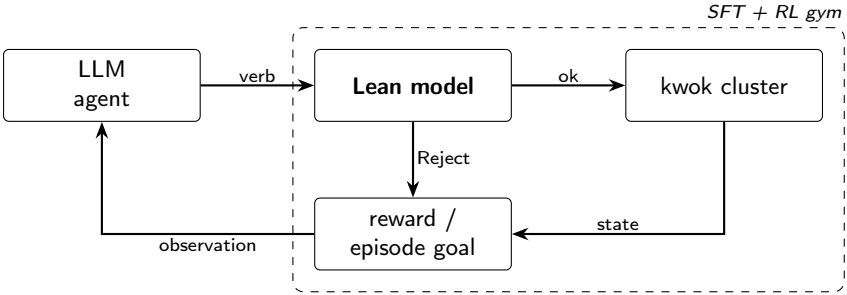
```
theorem admitSafe :
    admit s v = .ok () → safe (apply s v) = true
```

If `admit` returns `ok`, the resulting cluster state satisfies every registered invariant, for every state and every verb.

- It is not "the policy is correct because we reviewed it".
- The Lean kernel checks the proof; we only have to write the invariant.

One use-case of the model: a judge for an LLM SRE agent

The same Lean definitions are also a useful referee when training an agent that proposes operations on a cluster.



- Typed `Reject` values become typed reward features. No regex on error strings.
- Decidable goals like `drainNode`, `rolloutImage`, and `scaleTo` give clean episode termination.
- Anything past `admit` is, by theorem, on a safe trajectory.

Open questions

- Is this slice of the API the right one to model first?
- Which cluster invariants are missing from the registry?
- Is the synchronous `apply` faithful enough, or do we need an explicit controller step?
- Where else would a formal model of the cluster pay off, beyond admission and agent training?